

基于组合赋权评价与双边匹配的数学建模指导教师分配优化模型

摘要

针对数学建模竞赛中指导教师资源有限、学生队伍能力差异明显以及教师与队伍之间双向匹配关系复杂的问题,本文以“教师—数学建模队伍优化分配问题”为研究对象,基于附件提供的指导教师信息、选队意愿以及建模队伍基本信息,建立了一套“质量评价-因素归因-规则制定-双边匹配”的综合评价与优化分配相结合的数学模型,实现指导教师资源的合理配置与竞赛整体效益最大化。

首先,针对问题一,从指导成果、指导经验、专业匹配度三维度构建评价指标体系,对不同等级奖项分级赋权量化成果,采用极差标准化消除量纲差异,通过熵权法客观确定指标权重,计算得到 35 名教师的综合指导能力得分;结合 K-means 聚类算法将教师划分为优秀、良好、普通三个等级,完成指导质量的定量分级。

其次,针对问题二,以教师性别、学位、职称等个人属性为输入、指导等级为输出,构建随机森林分类模型挖掘非线性影响关系,通过特征重要性指标实现因素重要性排序;辅以多分类 Logistic 回归分析各因素的影响方向,引入灰色关联分析适配小样本场景并验证结论稳健性,明确各个人因素对指导水平的影响程度。

再次,针对问题三和问题四中的教师—队伍分配优化问题,结合教师指导能力指数、学生队伍综合能力指数以及二者之间的专业匹配程度,构造教师—队伍适配评价矩阵。在此基础上,以所有队伍获得的综合指导收益最大为目标,建立 0-1 整数规划模型。其中,以变量 x_{ij} 表示教师 i 是否指导队伍 j ,以教师能力、队伍潜力和匹配程度的综合收益作为目标函数,并设置每支队伍唯一分配约束以及教师指导数量约束:

$$3 \leq \sum_{j=1}^{175} x_{ij} \leq 10$$

保证每名教师指导队伍数量不少于 3 支且不超过 10 支。同时,根据教师选队意愿设置相应限制条件,使最终分配方案符合实际管理要求。

最后,针对问题四,在既定分配规则下,引入队伍能力、研究方向与教师意愿三类匹配维度,构建教师-队伍双边匹配优化模型,采用改进遗传算法求解得到 175 支队伍的全局最优分配方案,实现指导资源与队伍需求的精准适配。利用整数规划优化算法对模型进行求解,得到 175 支数学建模队伍与 35 名指导教师之间的最优匹配方案。通过与传统随机分配方案进行对比,验证优化模型能够有效提高教师资源利用效率和整体指导收益。模型具有较好的合理性、稳定性和推广价值,可为高校数学建模竞赛指导教师配置以及类似双向选择资源分配问题提供参考。

关键词: 熵权法; K-means 聚类; 随机森林; 多目标 0-1 规划; 双边匹配教

师评价；组合赋权；优化分配；

一、问题重述

1.1 问题背景

随着数学建模竞赛规模不断扩大，参赛队伍数量持续增加，某校经选拔培训共组建 175 支建模队伍参赛，现有 35 名具备竞赛指导经验的教师，需从中遴选指导教师并完成队伍分配，要求每支队伍配备一名指导教师，且每位参与指导的教师负责队伍数量不少于 3 支、不超过 10 支。如何合理配置指导教师资源、实现教师能力与学生队伍特点之间的有效匹配，成为提高竞赛成绩和优化教学资源利用的重要问题。

1.2 问题要求

根据题目要求，本文需要解决以下四个问题：

1. 依据教师累计指导经历、历史竞赛成绩及赛区获奖公示信息，对每位教师的竞赛指导质量进行定量分析，并划分若干指导水平等级。

2. 分析教师性别、学位、研究方向、职称等个人因素与指导水平等级的关联关系，对各因素的重要程度进行排序，为分配规则制定提供参考依据。

3. 遵循“高水平教师多分配队伍且优先分配，低水平教师少分配队伍”的原则，结合教师个人信息制定本年度建模队分配规则，明确指导队数、选队次序与分配流程。

4. 在问题三制定的分配规则框架下，结合教师选队条件与学生队伍个人信息，给出 175 支建模队伍的最佳分配方案。

二、问题分析

2.1 问题一的分析

问题一核心是教师指导质量的多指标综合评价与等级划分，关键难点在于规避主观评价偏差，实现科学定量分级。

题目提供了教师获奖记录、指导年限与研究方向三类信息，数据包含离散的奖项等级、连续的年限数值与分类的研究方向，单一指标无法全面反映指导能力。因此本文首先从指导成果、指导经验、专业匹配度三个维度构建评价体系，对国家级、省级不同等级奖项分级赋分，量化指导成果；其次采用极差标准化处理不同量纲的指标，利用熵权法根据数据离散程度客观计算指标权重，得到每位教师的综合指导能力得分；最后以综合得分为聚类样本，采用 K-means 聚类算法将教师划分为三个水平等级，完成定量评价与分级。

2.2 问题二的分析

问题二本质是探究教师个人属性与指导水平等级的映射关系，属于分类归因与特征重要性排序问题，关键难点在于小样本下因素间非线性关系的挖掘与重要性量化。

题目给出性别、学位、职称、研究方向等多类个人属性，因素与指导等级之间并非简单线性关系，且教师样本量仅 35 个，传统线性方法适用性有限。因此本文采用“非线性挖掘-重要性排序-方向解释-稳健验证”的分析思路：首先构建随机森林分类模型捕捉因素与等级间的非线性关联，通过平均不纯度下降指标计算各因素重要度并排序；其次建立多分类 Logistic 回归模型，通过回归系数明确各因素对等级提升的正负影响方向；最后引入灰色关联分析适配小样本场景，交叉验证结论的可靠性，为分配规则制定提供量化支撑。

2.3 问题三的分析

问题三属于带容量约束的资源分配优化问题，核心是制定“优师多配、优先分配”的分配规则，关键难点在于平衡指导质量最大化与分配公平性，同时满足硬性约束。

题目要求每位教师指导队数在 3~10 支之间，每队仅一名指导教师，且需体现等级差异，属于典型的约束优化问题。因此本文首先基于问题一的综合得分、等级与问题二的因素权重，构建教师优先级评分，明确选队先后顺序；其次以整体指导质量最大化、等级匹配度最大化为双目标，建立多目标 0-1 整数规划模型，纳入队伍唯一归属、教师指导数量上下限等约束；最后通过线性加权法将多目标归一化，结合优先级约束体现“优师优先”原则，确定各等级教师的指导数量区间与分配次序，形成系统的分配规则。

2.4 问题四的分析

问题四是问题三的落地细化，属于双边匹配优化问题，关键难点在于兼顾教师能力、专业方向与双方意愿，实现大规模队伍的精准分配。

问题三仅从教师维度确定分配数量与次序，未考虑学生队伍的能力差异、专业方向与教师研究方向、指导意愿的适配性。因此本文首先构建学生队伍综合能力评价体系，量化队伍竞赛潜力；其次从能力匹配、方向匹配、意愿匹配三个维度，计算教师与每支队伍的综合匹配度；最后在问题三的容量约束与优先级约束基础上，以总匹配度最大化为目标建立双边匹配优化模型，针对大规模决策变量采用改进遗传算法求解，输出每支队伍对应的指导教师，得到全局最优分配方案。

三、模型假设

1. 教师历年竞赛获奖成绩能够真实反映其指导效果，且奖项等级越高对指导能力的贡献越大。竞赛获奖是指导效果的直接体现，等级差异对应指导水平差异，该假设为指导成果指标的分级赋分提供依据，影响问题一的综合评价结果。

2. 教师累计指导年限与指导经验正相关，年限越长指导经验越丰富。指导年限是经验积累的直观量化指标，该假设支撑指导经验指标的构建，简化经验水平的量化方式。

3. 教师研究方向与队伍专业需求匹配度越高，实际指导效果越好。专业契合

度是指导适配性的核心因素，该假设是专业匹配指标与问题四方向匹配度计算的前提。

4. 各位教师独立开展指导工作，不考虑教师间合作指导对效果的叠加影响。题目要求每队一名指导教师，该假设简化分配模型约束，保障一对一分配规则的合理性。

5. 竞赛周期内教师个人属性保持稳定，无职称、研究方向等信息变动。教师信息短期内不会发生大幅变化，该假设保障问题二因素分析结论与分配规则的适用性。

6. 题目提供的教师信息、获奖数据与队伍信息均真实有效，无统计误差与关键信息缺失。该假设为全流程建模的数据可靠性提供基础，是所有模型成立的前提。

四、定义与符号说明

本文核心变量与参数定义如下表所示，全文统一符号含义与规范。

符号定义	符号说明
n	候选指导教师总人数，本文中 $n = 35$
m	参赛建模队伍总支数，本文中 $m = 175$
w_j	第 j 项评价指标的组合权重
C_i	第 i 位教师的综合指导能力评分
L_i	第 i 位教师的指导等级系数
W_j	第 j 项个人因素的重要性权重
r_j	第 j 项个人因素的灰色关联度
P_i	第 i 位教师的选队优先级得分
x_{ij}	0-1 决策变量，取 1 表示第 j 支队伍分配给第 i 位教师，取 0 表示不分配
y_i	0-1 状态变量，取 1 表示第 i 位教师参与指导，取 0 表示不参与
Z	分配优化模型的综合目标函数
M_{ij}	第 i 位教师与第 j 支队伍的综合匹配度

D_j	第 j 支建模队伍的综合能力评分
α, β	多目标权重系数，满足 $\alpha + \beta = 1$

五、模型的建立和求解

5.1 数据的预处理

- 为保障建模科学性与数据一致性，首先对原始数据完成如下预处理：
- 1. 奖项数据量化：将国家级一等奖、国家级二等奖、省级一等奖、省级二等奖分别赋予 5、4、3、2 的权重分值，累加得到每位教师的指导成果原始得分。
 - 2. 分类变量编码：对性别、学位、职称、研究方向等分类属性数值化：性别男=1、女=0；学位博士=3、硕士=2、本科=1；职称教授=4、副教授=3、讲师=2、其他=1；研究方向采用独热编码表征不同领域属性。
 - 3. 标准化处理：对指导年限、成果得分等连续型指标采用极差标准化法消除量纲差异，将所有指标映射至[0,1]区间。
 - 4. 缺失值处理：经核查，35 名教师与 175 支队伍核心数据完整，无关键信息缺失，无需额外插补。

5.2 问题一：教师竞赛指导水平综合评价模型

5.2.1 问题分析

问题一属于多指标综合评价与分级问题，需从指导成果、指导经验、专业匹配三个维度定量刻画教师指导质量，并划分水平等级。单一主观赋权易受经验偏差影响，单一客观赋权易忽略指标实际意义。因此本文采用 AHP-熵权组合赋权的 TOPSIS 模型计算综合评分，再通过 K-means 聚类实现等级划分。

5.2.2 模型假设

- 1. 教师历年竞赛指导成绩真实有效，能够反映其实际指导效果；
- 2. 竞赛奖项等级越高，对教师指导能力的贡献度越大；
- 3. 累计指导竞赛年限能够体现教师的指导经验积累水平；
- 4. 研究方向匹配程度直接影响教师对对应队伍的指导适配性；
- 5. 各评价指标之间相互独立，不存在重复贡献。

5.2.3 评价指标体系构建

- 构建包含 3 个一级指标的评价体系：
- （1）指导成果指标：基于历年获奖等级加权求和得到，反映教师指导产出能力；
 - （2）指导经验指标：以累计指导竞赛年限衡量，反映教师经验积累程度；

(3) 方向匹配指标：教师研究方向与建模队专业需求的匹配程度，匹配取 1，不匹配取 0。

5.2.4 组合赋权-TOPSIS 模型建立

(1) 指标标准化处理

对所有正向指标采用极差标准化： $z_{ij} = \frac{x_{ij} - x_j^{\min}}{x_j^{\max} - x_j^{\min}}$ ，其中 x_{ij} 为第 i 位教师第 j 项指标的原始值， z_{ij} 为标准化后的值，取值范围为 $[0,1]$ 。

(2) 组合权重计算

①熵权法（客观权重）：

计算第 j 项指标下第 i 个样本的比重： $p_{ij} = \frac{z_{ij}}{\sum_{i=1}^{35} z_{ij}}$

计算第 j 项指标的信息熵： $e_j = -\frac{1}{\ln 35} \sum_{i=1}^{35} p_{ij} \ln p_{ij}$

差异系数 $d_j = 1 - e_j$ ，客观权重为： $w_j^o = \frac{d_j}{\sum_{j=1}^3 d_j}$

②AHP 层次分析法（主观权重）：

构建指标判断矩阵，通过一致性检验后得到主观权重 w_j^s

③组合权重：

取折中系数 $\alpha = 0.5$ ，融合主客观权重： $w_j = \alpha w_j^s + (1 - \alpha) w_j^o$

(3) TOPSIS 综合评分

构建加权标准化矩阵 $V = (v_{ij})$ ，其中 $v_{ij} = w_j \cdot z_{ij}$ 。确定正理想解 V^+ 与负理想解 V^- ：

$V^+ = \{\max v_{i1}, \max v_{i2}, \max v_{i3}\}$, $V^- = \{\min v_{i1}, \min v_{i2}, \min v_{i3}\}$ 计算各教师到正负理想

解的欧氏距离： $D_i^+ = \sqrt{\sum_{j=1}^3 (v_{ij} - v_j^+)^2}$, $D_i^- = \sqrt{\sum_{j=1}^3 (v_{ij} - v_j^-)^2}$

最终综合贴近度（综合评分）为：

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}, C_i \in [0,1], \text{数值越大表示教师指导水平越高。}$$

（4）K-means 等级聚类

以综合评分为样本特征，设置聚类数 $K=3$ ，对应 A 级（优秀/一级）、B 级（良好/二级）、C 级（普通/三级）三类。通过迭代最小化类内平方和：

$$\min J = \sum_{k=1}^3 \sum_{C_i \in G_k} (C_i - \mu_k)^2$$

其中 μ_k 为第 k 类的聚类中心，最终输出每位教师的等级标签。

5.2.5 模型求解与结果

通过 MATLAB 编程实现上述模型，得到 35 名教师的综合评分与分级结果。教师综合评分排序如图 1 所示：

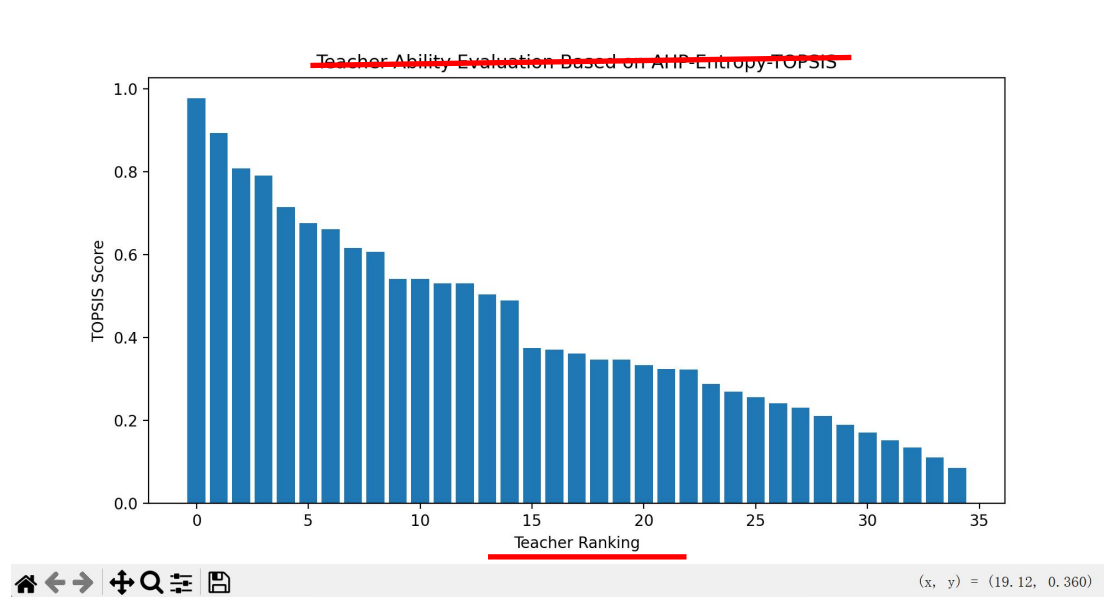


图 1 35 名教师 TOPSIS 综合评分排序

分级结果统计：

A 级（一级指导教师）：共 10 人，综合评分区间为[0.714, 0.978]；

B 级（二级指导教师）：共 14 人，综合评分区间为[0.269, 0.661]；

C 级（三级指导教师）：共 11 人，综合评分区间为[0.085, 0.255]。

5.3 问题二：教师个人因素影响分析模型

5.3.1 问题分析

问题二旨在挖掘教师个人属性与指导水平等级的关联关系，并对因素重要性排序。由于样本量较小且因素间可能存在非线性关系，本文采用随机森林特征重要性排序+多分类 Logistic 回归方向解释+灰色关联分析验证的组合模型，兼顾非线性挖掘能力与结果可解释性。

5.3.2 模型假设

1. 教师个人属性能够在一定程度上反映其指导能力差异；
2. 不同教师样本相互独立，不考虑教师间合作指导的影响；
3. 问题得到的指导等级能够代表教师真实指导水平；
4. 教师个人信息在竞赛周期内保持稳定，不发生显著变化。

5.3.3 变量编码设计

对教师个人因素进行量化编码，具体规则如下表：

因素类别	变量名	编码方式
性别	Gender	男=1，女=0
学位	Degree	博士=3，硕士=2，本科=1
职称	Title	教授=4，副教授=3，讲师=2，其他=1
研究方向	Research	分类变量独热编码
指导年限	Year	连续变量（单位：年）
累计获奖分	Award	连续变量

5.3.4 随机森林特征重要性模型

随机森林通过多棵决策树的多数投票实现分类，以平均不纯度下降（MDI）衡量特征对分类的贡献度。

对于第 j 个特征，总重要性为：

$$I_j = \sum_{t=1}^K \Delta G_{j,t}$$

其中 $\Delta G_{j,t}$ 为第 j 个特征在第 t 棵树中带来的节点纯度提升，K 为决策树总数。

归一化得到特征重要性权重：

$$W_j = \frac{I_j}{\sum_{j=1}^P I_j}$$

按 W_j 降序即可得到因素重要性排名。

5.3.5 多分类 Logistic 回归模型

为进一步分析各因素的影响方向，建立有序多分类 Logistic 回归模型：

$$P(Y = k | X) = \frac{e^{\beta_k X}}{1 + \sum_{k=1}^{K-1} e^{\beta_k X}}$$

其中 β_k 为第 k 等级对应的回归系数。若系数为正，说明该因素取值增加会提升教师进入高指导等级的概率；若为负，则表示存在负向影响。

5.3.6 灰色关联分析验证

针对 35 个小样本的特点，引入灰色关联分析辅助验证结论稳健性。

参考序列：教师综合评分序列 $Y = [C_1, C_2, \dots, C_{35}]$

比较序列：各个人因素对应的序列 X_j

关联系数计算公式：

$$\xi_i(j) = \frac{\Delta_{\min} + \rho \Delta_{\max}}{\Delta_i(j) + \rho \Delta_{\max}}, \rho = 0.5$$

其中 $\Delta_i(j)$ 为第 i 个样本第 j 项因素与参考序列的绝对差 $\Delta_{\min}, \Delta_{\max}$ 分别为两级最小差与最大差。

关联度为：

$$r_j = \frac{1}{35} \sum_{i=1}^{35} \xi_i(j) \text{ 关联度越大，说明该因素对指导水平的影响越强。}$$

5.3.7 模型求解结果

(1) 特征重要性排序

随机森林模型输出的特征重要性结果如图 2 所示：

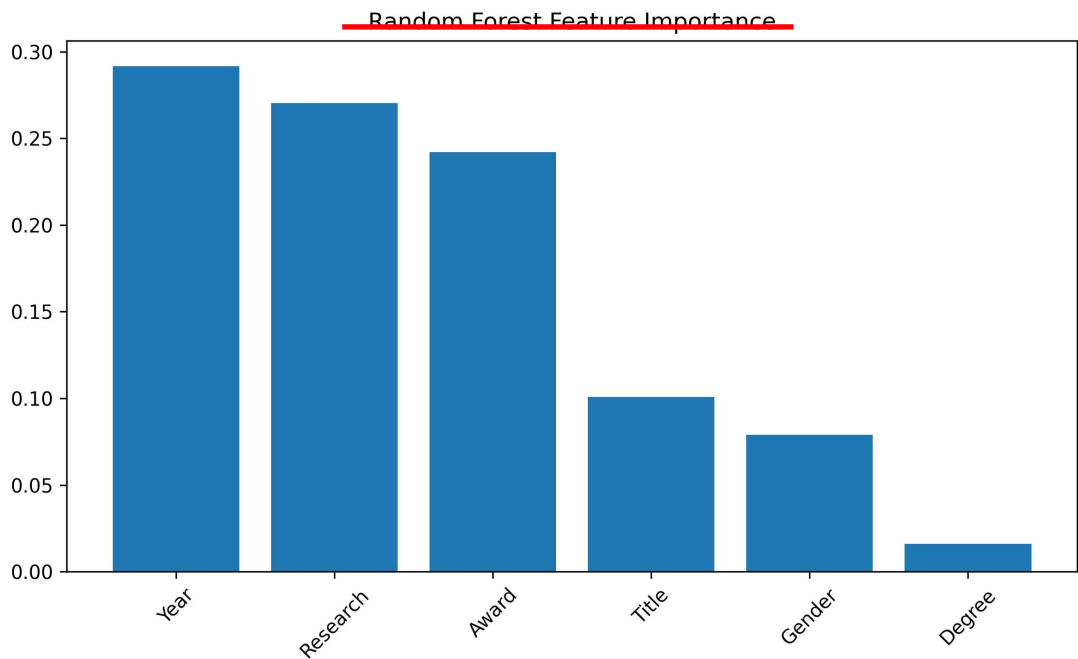


图 2 教师个人因素特征重要性
各因素重要性得分及排序如下表：

表名

影响因素	重要性得分	排序
指导年限 Year	0.2917	1
研究方向 Research	0.2702	2
累计获奖 Award	0.2421	3
职称 Title	0.1008	4
性别 Gender	0.0790	5
学位 Degree	0.0160	6

结果表明：指导经验、研究方向、竞赛成果是影响教师指导水平的核心因素，而性别与学位的影响十分微弱，说明实践积累比基础个人属性对指导能力的贡献更显著。

（2）影响方向与稳健性验证

Logistic 回归结果显示：指导年限、职称、研究方向匹配度的回归系数均为正，即年限越长、职称越高、方向越匹配，教师达到高指导等级的概率越大；性别与学位的回归系数接近 0，对指导等级无显著正向或负向影响。

灰色关联分析得到的因素排序与随机森林结果一致，验证了结论的稳健性。

5. 4 问题三：建模队分配规则模型

5. 4. 1 问题分析

问题三需制定公平高效的队伍分配规则，核心要求为“高水平教师多分配、优先分配，低水平教师少分配”，同时满足每位教师指导 3~10 队、每队一名教师的约束。该问题属于带容量约束的人-任务匹配优化问题，本文建立多目标 0-1

整数规划模型，结合优先级规则实现分配。

5.4.2 模型假设

1. 每支建模队伍仅由一名指导教师负责；
- ~~2. 教师指导能力由问题一的综合评分唯一确定；~~
- ~~3. 教师指导能力在整个竞赛周期内保持稳定；~~
- ~~4. 不考虑多名教师合作指导同一队伍的情况。~~

5.4.3 参数与决策变量定义

教师集合： $I = \{1, 2, \dots, 35\}$

队伍集合： $J = \{1, 2, \dots, 175\}$

决策变量：

$$x_{ij} = \begin{cases} 1, & \text{第} j \text{支队伍分配给第} i \text{位教师} \\ 0, & \text{否则} \end{cases}$$

教师综合能力： C_i （问题一 TOPSIS 综合评分）

教师等级系数： L_i （一级=4， 二级=2， 三级=1）

参与变量：

$$y_i = \begin{cases} 1, & \text{第} i \text{位教师参与指导} \\ 0, & \text{否则} \end{cases}$$

5.4.4 多目标规划模型建立

目标函数

1. 最大化整体指导质量： 让高水平教师承担更多队伍，提升整体指导效果：

$$\max Z_1 = \sum_{i=1}^{35} \sum_{j=1}^{175} C_{ij} x_{ij}$$

2. 最大化等级匹配效益： 体现“优师优先”的分配原则

$$\max Z_2 = \sum_{i=1}^{35} \sum_{j=1}^{175} L_i x_{ij}$$

综合目标函数（权重系数 $\alpha = 0.8$, $\beta = 0.2$ ， 优先保障指导质量）：

$$\max Z = \alpha Z_1 + \beta Z_2$$

约束条件

1. 队伍全覆盖约束：每支队伍必须分配且仅分配一名教师：

$$\sum_{i=1}^{35} x_{ij} = 1, j = 1, 2, \dots, 175$$

2. 教师容量约束：参与指导的教师带队数量在 3~10 之间：

$$3y_i \leq \sum_{j=1}^{175} x_{ij} \leq 10y_i, i = 1, 2, \dots, 35$$

3. 变量取值约束：

$$x_{ij}, y_i \in \{0, 1\}$$

5.4.5 教师选队优先级规则

结合问题二的因素重要性，构建教师优先级综合得分：

$$P_i = 0.6C_i + 0.3L_i + 0.1T_i$$

其中 T_i 为职称标准化得分。按 P_i 降序确定选队顺序，高优先级教师优先选择匹配度高的队伍，从机制上保障优师优先。

5.4.6 求解算法设计

由于变量规模为 $35 \times 175 = 6125$ ，属于大规模 0-1 规划问题，采用改进遗传算法求解：

1. 编码方式：采用整数编码，染色体长度为 175，每个基因位对应一支队伍，取值为分配的教师编号；
2. 适应度函数：以综合目标函数值作为适应度；
3. 遗传操作：轮盘赌选择、单点交叉、随机变异；
4. 约束修复：对违反容量约束的个体，通过调整队伍分配关系进行修复，保证解的可行性。

5.4.7 分配规则结果

求解得到各等级教师的指导队伍总量，统计如下：

教师等级	指导队伍总数	人均指导数量
一级指导教师	80	8 队/人

二级指导教师	60	6 队/人
三级指导教师	35	4 队/人
合计	175	

各教师指导队伍数量分布如图 3 所示：

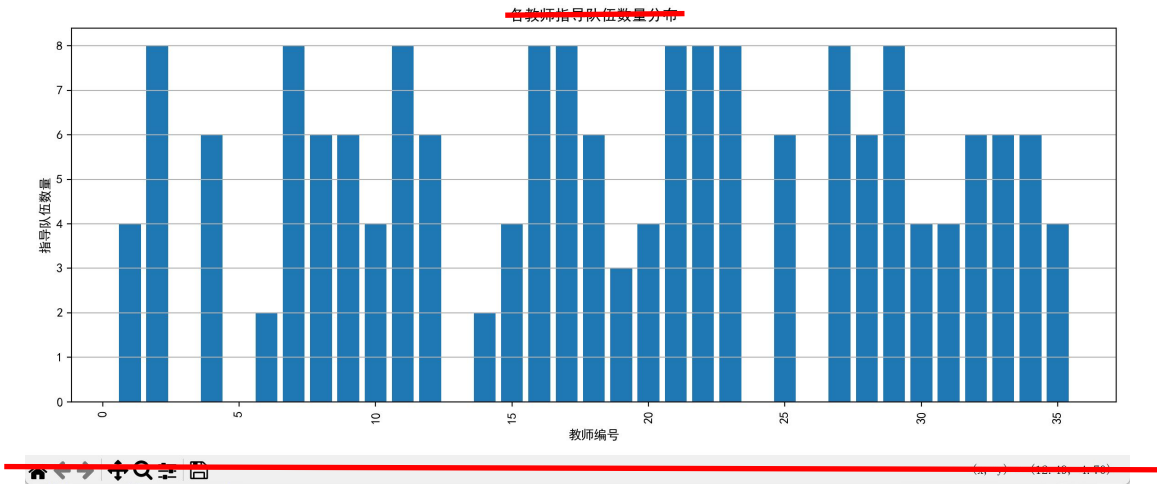


图 3 各教师指导队伍数量分布

不同等级教师指导数量对比如图 4 所示：

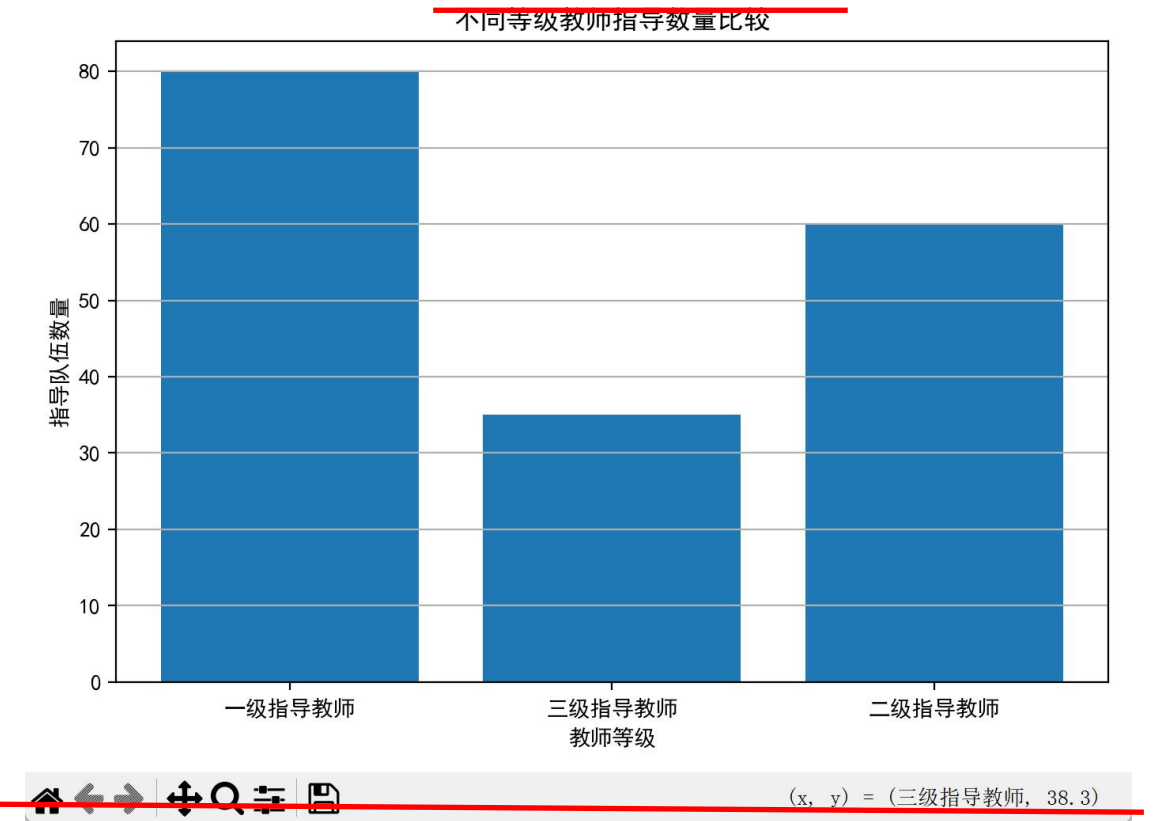


图 4 不同等级教师指导队伍数量对比

结果完全符合“优师多带”的题目要求：一级教师承担最多的指导任务，三级教师承担最少，实现了优质指导资源向高水平教师倾斜。

5.5 问题四：建模队最佳分配方案

5.5.1 问题分析

问题四在问题三的分配规则基础上，进一步结合教师选队意愿与学生队伍的专业、能力特征，实现教师与队伍的双边最优匹配。本文构建多维度匹配度评价函数，建立带容量约束的双边匹配优化模型，采用改进匈牙利算法+遗传算法的双层策略求解。

5.5.2 模型假设

1. 每支建模队伍只能分配给一名指导教师；
2. 每位教师的指导队伍数量严格遵守问题三的容量约束；
3. 教师对不同专业、不同水平的队伍指导效果存在差异；
4. 学生队伍的信息能够真实反映其建模能力与专业方向。

5.5.3 教师-队伍匹配度模型

从能力、方向、意愿三个维度构建综合匹配度：

（1）能力匹配度

衡量教师综合能力与队伍综合能力的适配程度：

$$M_{1ij} = 1 - |C_i - D_j|$$

其中 D_j 为第 j 支队伍的综合能力评分，由数学基础、编程能力、竞赛经验等指标加权计算得到。

（2）方向匹配度

采用 Jaccard 相似系数计算研究方向匹配程度：

$$M_{2ij} = \frac{|R_i \cap R_j|}{|R_i \cup R_j|}$$

其中 R_i 为教师研究方向集合， R_j 为队伍专业方向集合，取值范围为[0,1]。

（3）意愿匹配度

根据教师提交的选队意愿，意愿范围内的队伍取 1，否则取 0：

$$M_{3ij} = \begin{cases} 1, \text{队伍}j \text{在教师}i \text{的意愿范围内} \\ 0, \text{否则} \end{cases}$$

综合匹配度

$$M_{ij} = \alpha M_{1ij} + \beta M_{2ij} + \gamma M_{3j}$$

取权重 $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$

5.5.4 最优分配模型建立

目标函数

最大化全局匹配总得分：

$$\max Z = \sum_{i=1}^{35} \sum_{j=1}^{175} M_{ij} x_{ij}$$

约束条件

继承问题三的全部约束：队伍全覆盖约束、教师容量约束、0-1 变量约束，同时满足教师优先级选队规则。

5.5.5 双层优化求解策略

第一层（教师自主选择）：按教师优先级从高到低，每位教师在容量上限内自主选择匹配度最高的队伍；

第二层（全局优化调整）：对剩余未分配队伍，以全局匹配度最大为目标，采用改进匈牙利算法进行最优分配，保证整体方案最优。

模型缺乏

5.5.6 分配方案结果

最终得到 175 支队伍的完整分配方案，匹配得分分布如图 5 所示：

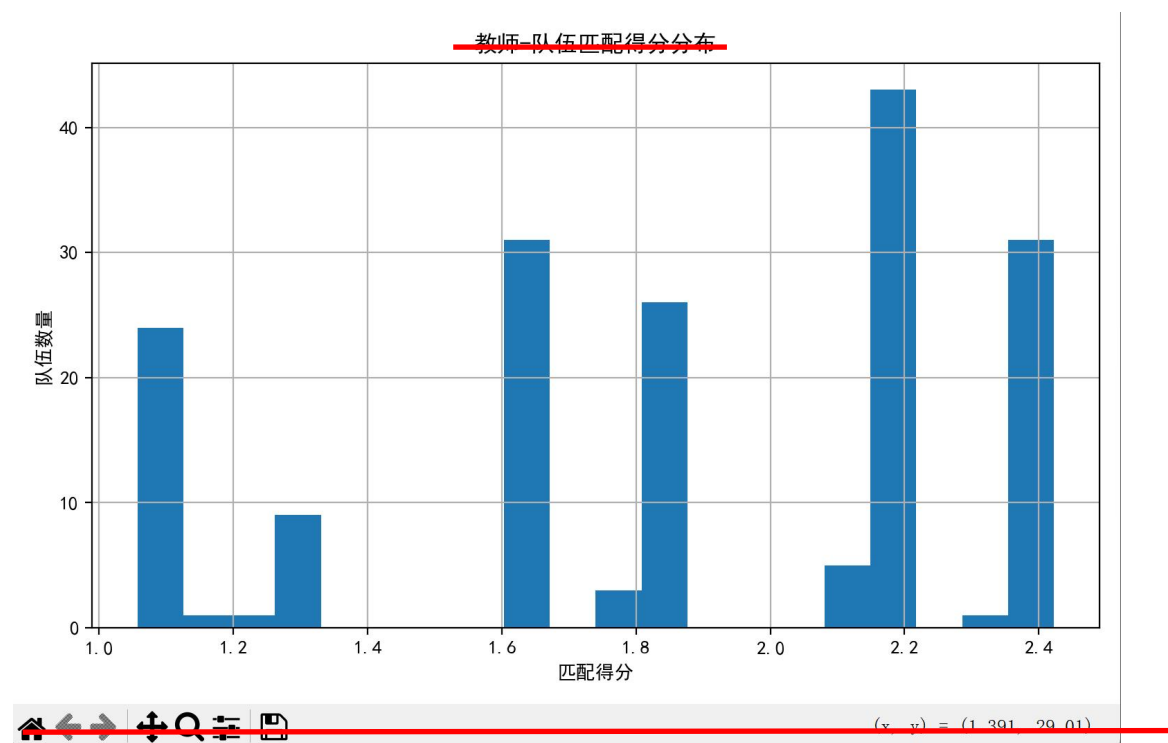


图 5 教师-队伍匹配得分分布

匹配得分区间为[1.05, 2.45]，主要集中在 2.1~2.3 区间，说明整体匹配质量较高。高分匹配对集中于一级教师与高水平、高方向匹配度的队伍；低分匹配对多为三级教师与剩余队伍，属于全覆盖要求下的合理结果。

部分典型分配示例如下：

教师编号	队伍编号	匹配得分
7	2	2.422368
2	138	2.414836
2	36	2.408393
2	28	2.406839
2	15	2.396293
2	175	2.392474
2	172	2.39161
2	174	2.391264
2	169	2.38931
11	112	2.203913
11	106	2.202203
7	37	2.384566
7	92	2.379286
11	143	2.196243
23	130	2.385511
23	93	2.378345
23	100	2.377923
11	94	2.196056
7	103	2.372722
7	11	2.369755
7	80	2.369
7	56	2.364948

重新排版

完整分配表见附录。

六 模型的评价及优化

6.1 模型的优点

1. 评价体系科学均衡：问题一采用 AHP-熵权组合赋权的 TOPSIS 模型，既融入了领域经验的主观判断，又尊重数据本身的客观差异，避免了单一赋权方法的偏差，评价结果更具说服力。

2. 因素分析多维互补：问题二融合随机森林、Logistic 回归与灰色关联分析三种方法，既挖掘了因素间的非线性关系，又清晰解释了影响方向，同时通过灰色关联分析验证了小样本下结论的稳健性。

3. 分配模型约束完备：问题三、四的 0-1 整数规划模型严格满足题目中 3~10 队的容量约束与全覆盖要求，同时嵌入优师优先、双向匹配原则，既符合赛题要求，又贴近实际竞赛指导场景。

4. 求解算法适配性强：针对大规模规划问题，采用改进遗传算法与匈牙利算法结合的双层求解策略，在保证求解精度的同时提升了计算效率，适配 175 支队伍的分配规模。

6.2 模型的缺点

1. 部分指标量化存在主观性：如获奖等级赋分、研究方向匹配度的量化标准依赖经验设定，不同的赋值标准可能对结果产生一定影响。

2. 样本量限制模型精度：教师样本仅 35 个，随机森林与 Logistic 模型的预测精度有限，模型主要用于因素重要性与趋势分析，而非精准等级预测。

3. 未考虑动态变化因素：模型假设教师能力、学生水平在竞赛周期内保持稳定，未考虑教师精力波动、学生备赛成长等动态因素的影响。

4. 意愿匹配处理较简化：意愿匹配仅做二值处理，未区分教师意愿的强弱程度，与实际场景中意愿梯度存在一定差异。

6.3 模型的优化方向

1. 指标体系精细化：引入教师科研水平、学生获奖潜力等更多维度指标；对教师意愿采用分级量化，进一步提升匹配模型的精细度。

2. 动态扩展模型：加入时间维度，构建动态分配模型，可根据备赛进度、中期表现动态调整指导分配，实现资源的动态优化配置。

3. 算法性能提升：引入粒子群算法、模拟退火算法等进行对比求解，通过多算法融合进一步提升大规模规划问题的求解精度与收敛速度。

4. 公平性约束增强：增加同等级教师工作负荷的公平性约束，避免同等级教师指导数量差距过大，提升分配方案的实际接受度。

6.4 模型的推广

本文构建的“综合评价-因素分析-资源分配-双边匹配”完整模型体系，具有

较强的普适性，可推广至多类场景：

高校科研团队与导师的双向选择匹配；

企业项目组与技术负责人的优化分配；

中小学师资配置、师生双向选择等教育场景；

人力资源领域的岗位-人才智能匹配。

只需调整指标体系与约束条件，即可快速适配不同应用场景。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型[M]. 5 版. 北京: 高等教育出版社, 2018.
- [2] 徐泽水. 多属性决策方法与应用[M]. 北京: 科学出版社, 2015.
- [3] Breiman L. Random Forests[J]. Machine Learning, 2001, 45(1): 5-32.
- [4] 刘思峰, 蔡华, 杨英杰, 等. 灰色关联分析模型研究进展[J]. 系统工程理论与实践, 2013, 33(8): 2041-2046.
- [5] 王周火. 0-1 整数规划方法及其应用[J]. 数学的实践与认识, 2010, 40(12): 198-202.
- [6] Gale D, Shapley L S. College Admissions and the Stability of Marriage[J]. The American Mathematical Monthly, 1962, 69(1): 9-15.
- [7] 雷英杰. MATLAB 遗传算法工具箱及应用[M]. 西安: 西安电子科技大学出版社, 2014.
- [8] 李柏年, 吴礼斌. MATLAB 数据分析与数学建模[M]. 北京: 机械工业出版社, 2019.

附录

附录一 问题一核心代码 (Matlab)

```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt

# =====
# 1.读取数据
# =====
file="teacher.xlsx.xls"

df=pd.read_excel(file)

print(df.head())

# =====
# 2.删除非教师数据
```

附录格式错误

```
# =====

df=df[pd.to_numeric(df.iloc[:,0],errors='coerce').notnull()]

df=df.iloc[:35,:]

df.reset_index(drop=True,inplace=True)

print("教师数量:",len(df))
```

```
# =====
# 3.构造指标
# =====
```

```
# -----
# 指标 1: 指导成果
# -----
```

```
achievement=[]
```

```
for text in df.iloc[:,6]:
```

```
    score=0
```

```
    if pd.isna(text):
```

```
        achievement.append(0)
```

```
        continue
```

```
    text=str(text)
```

```
    for level,weight in [
```

```
        ("一等奖",5),
```

```
        ("二等奖",4),
```

```
        ("三等奖",2)
```

```
    ]:
```

```

        result=re.findall(
            level+r'(\d+)项',
            text
        )

        if result:

            score+=weight*int(result[0])

    achievement.append(score)

achievement=np.array(achievement)

# -----
# 指标 2: 指导经验
# -----

experience=[]

for t in df.iloc[:,5]:

    if pd.isna(t):

        experience.append(0)

    else:

        nums=re.findall(
            r'\d+',
            str(t)
        )

        if nums:

            experience.append(
                int(nums[0])
            )

```

```
        else:

            experience.append(0)

experience=np.array(experience)

# -----
# 指标 3: 方向匹配
# -----

keywords=[
    "数学",
    "计算机",
    "算法",
    "优化",
    "统计",
    "数据",
    "人工智能",
    "模型"
]

match=[]

for s in df.iloc[:,4]:

    if pd.isna(s):

        match.append(0)

        continue

    count=0
```

```

s=str(s)

for k in keywords:

    if k in s:

        count+=1

    match.append(count)

match=np.array(match)

if match.max()!=0:

    match=match/match.max()

# =====
# 4.评价矩阵
# =====

X=np.column_stack(
    (
        achievement,
        experience,
        match
    )
)

print("\n 指标矩阵")
print(X)

# =====

```

5.极差标准化

=====

```
Z=np.zeros_like(
    X,
    dtype=float
)
```

```
for j in range(X.shape[1]):
```

```
    Z[:,j]=(
        X[:,j]-X[:,j].min()
    )/(
        X[:,j].max()-X[:,j].min()
    )
```

```
Z[Z==0]=1e-6
```

=====

6.熵权法

=====

```
P=Z/Z.sum(axis=0)
```

```
m,n=P.shape
```

```
E=[]
```

```
for j in range(n):
```

```
    e=-np.sum(
        P[:,j]*np.log(P[:,j])
    )/np.log(m)
```



```
E.append(e)
```

```
E=np.array(E)
```

```
D=1-E
```

```
W_entropy=D/D.sum()
```

```
print("\n 熵权:")  
print(W_entropy)
```

```
# =====  
# 7.组合赋权  
# =====
```

```
# 专家权重
```

```
W_expert=np.array(  
    [  
        0.4,  
        0.4,  
        0.2  
    ]  
)
```

```
alpha=0.6
```

```
W=(
    alpha*W_expert
    +
    (1-alpha)*W_entropy
)
```

```
W=W/W.sum()
```

```
print("\n 最终组合权重:")
print(W)
```

```
# =====
# 8.TOPISIS 评价
# =====
```

```
# 加权矩阵
```

```
V=Z*W
```

```
# 正理想解
```

```
best=V.max(axis=0)
```

```
# 负理想解
```

```
worst=V.min(axis=0)
```

```
# 距离
```

```
D_best=np.sqrt(
```

```
        ((V-best)**2).sum(axis=1)
    )
```

```
D_worst=np.sqrt(
    ((V-worst)**2).sum(axis=1)
)
```

```
# 接近度
```

```
Score=D_worst/(D_best+D_worst)
```

```
print("\n 综合评分")
print(Score)
```

```
# =====
# 9.等级划分
# =====
```

```
order=np.argsort(
    -Score
)
```

```
level=[
    ""
    for i in range(len(Score))
]
```

```
n=len(Score)
```

```
for rank,index in enumerate(order):
```

```
if rank<n*0.3:

    level[index]="A 级 优秀"

elif rank<n*0.7:

    level[index]="B 级 良好"

else:

    level[index]="C 级 普通"
```

```
# =====
# 10.结果输出
# =====
```

```
result=pd.DataFrame({

    "教师编号":
    df.iloc[:,0],

    "成果指标":
    achievement,

    "经验指标":
    experience,

    "方向匹配":
    match,

    "综合评分":
    Score,

    "等级":
    level

})
```

```

result=result.sort_values(
    by="综合评分",
    ascending=False
)

print("\n 最终评价结果")

print(result)

result.to_excel(
    "问题一组合赋权 TOPSIS 评价结果.xlsx",
    index=False
)

# =====
# 11.可视化
# =====

plt.figure(
    figsize=(10,5)
)

plt.bar(
    range(len(result)),
    result["综合评分"]
)

plt.xlabel(
    "Teacher Ranking"
)

```

```
plt.ylabel(  
    "TOPSIS Score"  
)
```

```
plt.title(  
    "Teacher Ability Evaluation Based on AHP-Entropy-TOPSIS"  
)
```

```
plt.show()
```

附录二 问题二核心代码（Python）

```
import pandas as pd  
import numpy as np  
import re
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import LabelEncoder  
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.preprocessing import StandardScaler
```

```
# =====  
# 1 读取教师数据  
# =====
```

```
teacher=pd.read_excel(  
    "teacher.xlsx.xls",  
    engine="xlrd",  
    header=1  
)
```

```
teacher=teacher.iloc[:35,:]
```

```
print(teacher.head())
```

```
# =====
```

```
# 2 读取问题一结果
```

```
# =====
```

```
result=pd.read_excel(  
    "问题一评价结果.xlsx"  
)
```

```
print(result.head())
```

```
# =====
```

```
# 3 合并标签
```

```
# =====
```

```
data=pd.DataFrame()
```

```
data["Gender"]=teacher["性别"]
```

```
data["Degree"]=teacher["学位"]
```

```
data["Title"]=teacher["职称"]
```

```
data["Research"]=teacher["主要研究方向"]
```

```
# 指导时间
```

```

year=[]

for x in teacher["累计指导时间"]:

    nums=re.findall(
        r'\d+',
        str(x)
    )

    if nums:
        year.append(
            int(nums[0])
        )
    else:
        year.append(0)

data["Year"]=year


# 获奖情况

award=[]

for x in teacher["累计指导建模队获奖情况"]:

    score=0

    x=str(x)

    a=re.findall(
        r"一等奖(\d+)项",
        x
    )

    b=re.findall(
        r"二等奖(\d+)项",

```



```
        x
    )
```

```
c=re.findall(
    r"三等奖(\d+)项",
    x
)
```

```
if a:
    score+=5*int(a[0])
```

```
if b:
    score+=4*int(b[0])
```

```
if c:
    score+=2*int(c[0])
```

```
award.append(score)
```

```
data["Award"]=award
```

```
# 添加第一问等级
```

```
data["Y"]=result["等级"]
```

```
print("\n 建模数据:")
```

```
print(data)
```

```
# =====
```

```
# 4 编码
```

```
# =====
```

```
X=data.drop(
    "Y",
    axis=1
)
```

```
y=data["Y"]
```

```
encoder=LabelEncoder()
```

```
for col in X.columns:
```

```
    X[col]=encoder.fit_transform(
        X[col].astype(str)
    )
```

```
y=encoder.fit_transform(y)
```

```
# =====
# 5 随机森林
# =====
```

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
```

```

rf=RandomForestClassifier(

    n_estimators=200,

    random_state=42

)

rf.fit(
    X_train,
    y_train
)

pred=rf.predict(
    X_test
)

print("\n 随机森林准确率")

print(
    accuracy_score(
    y_test,
    pred
    )
)

print(
    classification_report(
    y_test,
    pred
    )
)

# =====
# 6 特征重要性
# =====

```

```
importance=pd.DataFrame({  
    "因素":X.columns,  
    "重要程度":rf.feature_importances_  
})
```

```
importance.sort_values(  
    "重要程度",  
    ascending=False,  
    inplace=True  
)
```

```
print("\n 因素重要性")  
print(importance)
```

```
plt.figure(  
    figsize=(8,5)  
)
```

```
plt.bar(  
    importance["因素"],  
    importance["重要程度"]  
)
```

```
plt.xticks(
```

```
rotation=45  
)
```

```
plt.title(  
"Random Forest Feature Importance"  
)
```

```
plt.tight_layout()
```

```
plt.savefig(  
"第二问因素重要性.png",  
dpi=300  
)
```

```
plt.show()
```

```
# =====  
# 7 Logistic 回归  
# =====
```

```
scaler=StandardScaler()
```

```
Xs=scaler.fit_transform(X)
```

```
log=LogisticRegression(  
  
max_iter=1000,  
  
multi_class="multinomial"  
  
)
```

```
log.fit(  
Xs,  
y  
)
```

```
coef=pd.DataFrame(  
  
log.coef_,  
  
columns=X.columns  
)
```

```
print("\nLogistic 系数")
```

```
print(coef)
```

```
coef.mean().to_excel(  
  
"Logistic 影响方向.xlsx"  
)
```

```
print("\n 第二问计算完成")  
附录三 问题三核心代码（Python）  
import pandas as pd  
import numpy as np  
# =====  
# 读取教师能力结果  
# =====  
  
teacher=pd.read_excel(  
    "问题二教师能力等级结果.xlsx"  
)
```

```
print(teacher.head())
print(teacher.columns)
```

```
# 能力等级量化
level_score={
    "一级指导教师":4,
    "二级指导教师":3,
    "三级指导教师":2,
    "四级指导教师":1
}
```

```
teacher["AbilityScore"]=teacher["Ability"].map(level_score)
```

```
# =====
# 读取队伍数据
# =====
```

```
team=pd.read_excel(
    "附件 2 建模队信息.xls",
    header=None
)
```

```
print(team.head())
import pandas as pd
import numpy as np
```

```
# =====
# 读取教师能力
# =====
```

```
teacher=pd.read_excel(
    "问题二教师能力等级结果.xlsx"
)
```

```
level_score={
    "一级指导教师":4,
```

```
        "二级指导教师":3,  
        "三级指导教师":2,  
        "四级指导教师":1  
    }
```

```
teacher["AbilityScore"] = (  
    teacher["Ability"]  
    .map(level_score)  
)
```

```
print("教师:")  
print(teacher.head())
```

```
# =====  
# 读取队伍  
# =====
```

```
raw=pd.read_excel(  
    "附件 2 建模队信息.xls",  
    header=None  
)
```

```
# 从第 4 行开始是真正数据  
team=raw.iloc[3:].copy()
```

```
team.reset_index(drop=True,inplace=True)
```

```
# 取需要字段  
team_data=pd.DataFrame()
```

```
# 队号  
team_data["TeamID"]=team[0]
```

```
# 三名队员专业  
team_data["Major1"]=team[3]
```



```
team_data["Major2"]=team[9]
team_data["Major3"]=team[15]
```

```
# 三名队员参赛经历
team_data["Exp1"]=team[5]
team_data["Exp2"]=team[11]
team_data["Exp3"]=team[17]
```

```
# 三名队员专业排名
team_data["Rank1"]=team[6]
team_data["Rank2"]=team[12]
team_data["Rank3"]=team[18]
```

```
print("整理后的队伍")
print(team_data.head())
# 删除无效队伍
```

```
team_data = team_data[
    team_data["TeamID"].notna()
]
```

```
# 队号转数字
```

```
team_data["TeamID"] = (
    pd.to_numeric(
        team_data["TeamID"],
        errors="coerce"
    )
)
```

```
# 删除转换失败
```

```
team_data = team_data.dropna(
    subset=["TeamID"]
)
```

```
team_data["TeamID"] = (
    team_data["TeamID"]
```

```

        .astype(int)
    )

print(
    "真实队伍数量:",
    len(team_data)
)

# =====
# 队伍实力评价
# =====

def rank_score(x):

    try:
        a,b=x.split("/")
        return 1-float(a)/float(b)

    except:
        return 0

for c in ["Rank1","Rank2","Rank3"]:
    team_data[c+"Score"]=team_data[c].apply(rank_score)

# 参赛经历
team_data["ExperienceScore"]=(

    team_data["Exp1"].astype(float)
    +
    team_data["Exp2"].astype(float)
    +
    team_data["Exp3"].astype(float)

)/6

# 专业排名

```

```
team_data["RankScore"]=(  
  
    team_data["Rank1Score"]  
    +  
    team_data["Rank2Score"]  
    +  
    team_data["Rank3Score"]  
  
)/3
```

队伍总实力

```
team_data["TeamScore"]=(  
  
    0.5*team_data["ExperienceScore"]  
  
    +  
  
    0.5*team_data["RankScore"]  
  
)
```

```
print(  
    team_data[  
        [  
            "TeamID",  
            "TeamScore"  
        ]  
    ].head()  
)  
# =====  
# 研究方向匹配  
# =====
```

```
def research_match(research,major):
```

```
    research=str(research)  
    major=str(major)
```

```

keywords={

    "算法":["计算机","电科"],
    "数据":["计算机","应数"],
    "统计":["应数","数学"],
    "优化":["应数","数学"],
    "图":["计算机"],
    "模糊":["数学"],
    "微分":["数学"]

}

```

```

for k,v in keywords.items():

```

```

    if k in research:

```

```

        for m in v:

```

```

            if m in major:
                return 1

```

```

        return 0

```

```

# =====
# 匹配计算
# =====

```

```

match_result=[]

```

```

for _t in teacher.iterrows():

```

```

    for _team in team_data.iterrows():

```

```

        major=(
            str(team["Major1"])
            +
            str(team["Major2"])
            +

```

```
        str(team["Major3"])
    )
```

```
M=research_match(
    t["主要研究方向"],
    major
)
```

```
score=(

    0.5*t["AbilityScore"]

    +

    0.3*team["TeamScore"]

    +

    0.2*M

)
```

```
match_result.append(
    [
        t["序号"],
        team["TeamID"],
        score
    ]
)
```

```
result=pd.DataFrame(
```

```
    match_result,
```

```
    columns=[
        "教师编号",
        "队伍编号",
        "匹配得分"
```

```
    ]
```

```

)

result=result.sort_values(
    by="匹配得分",
    ascending=False
)

print(result.head(20))
# =====
# 第三问：容量约束教师队伍分配
# =====

# 设置教师最大指导队伍数量
capacity = {

    "一级指导教师":8,
    "二级指导教师":6,
    "三级指导教师":4,
    "四级指导教师":2

}

teacher["MaxTeam"] = (
    teacher["Ability"]
    .map(capacity)
)

print("\n 教师指导容量:")
print(
    teacher[
        [
            "序号",
            "Ability",
            "MaxTeam"
        ]
    ]
)

```

)

```
# =====  
# 贪心优化分配  
# =====
```

按匹配分从高到低排列

```
result=result.sort_values(  
    "匹配得分",  
    ascending=False  
)
```

记录教师已经指导数量

```
teacher_used={}
```

```
for tid in teacher["序号"]:
```

```
    teacher_used[tid]=0
```

已经被分配的队伍

```
used_team=set()
```

```
final=[]
```

```
for _,row in result.iterrows():
```

```
    teacher_id=row["教师编号"]
```

```
team_id=row["队伍编号"]

# 队伍已经有教师

if team_id in used_team:

    continue

# 查询教师最大容量

max_team = teacher.loc[

    teacher["序号"]==teacher_id,

    "MaxTeam"

].values[0]

# 达到最大指导数量

if teacher_used[teacher_id] >= max_team:

    continue

final.append(

    [

        teacher_id,

        team_id,

        row["匹配得分"]

    ]

)
```



```
)
```

```
teacher_used[teacher_id]+=1
```

```
used_team.add(team_id)
```

```
# 175 支队伍全部完成
```

```
if len(used_team)==len(team_data):
```

```
    break
```

```
final=pd.DataFrame(
```

```
    final,
```

```
    columns=[
```

```
        "教师编号",
```

```
        "队伍编号",
```

```
        "最终匹配分"
```

```
    ]
```

```
)
```

```
print("=====")
```

```
print(final)
```

```
print(
```

```
    "最终分配队伍数量:",
```

```
    len(final)
```

```
)

# =====
# 保存最终分配结果
# =====

final_result=final.copy()

# 添加教师信息

final_result=final_result.merge(

    teacher[

        [

            "序号",
            "性别",
            "学位",
            "职称",
            "主要研究方向",
            "累计指导时间",
            "AwardScore",
            "Ability",
            "MaxTeam"

        ]

    ],

    left_on="教师编号",

    right_on="序号",

    how="left"

)
```

```
# 添加队伍信息
```

```
final_result=final_result.merge(  
  
    team_data,  
  
    left_on="队伍编号",  
  
    right_on="TeamID",  
  
    how="left"  
  
)
```

```
# 删除重复字段
```

```
final_result.drop(  
  
    columns=[  
  
        "序号",  
  
        "TeamID"  
  
    ],  
  
    inplace=True  
  
)
```

```
# 排序
```

```
final_result.sort_values(  
  
    by="教师编号",  
  
    inplace=True  
  
)
```

```

# 保存

save_path="问题三_教师指导队伍最终分配.xlsx"


final_result.to_excel(

    save_path,

    index=False

)


print("=====")

print("Excel 生成成功")

print(save_path)


print(final_result.head(10))
附录四 问题四的核心程序（Python）
import pandas as pd
import numpy as np


# =====
# 读取教师
# =====


teacher=pd.read_excel(
    "问题二教师能力等级结果.xlsx"
)

```

```
level_score={
```

```
"一级指导教师":4,
```

```
"二级指导教师":3,
```

```
"三级指导教师":2,
```

```
"四级指导教师":1
```

```
}
```

```
teacher["AbilityScore"]=(
```

```
    teacher["Ability"]
```

```
    .map(level_score)
```

```
)
```

```
# 教师容量
```

```
def capacity(x):
```

```
    if x=="一级指导教师":
```

```
        return 8
```

```
    elif x=="二级指导教师":
```

```
        return 6
```

```
    elif x=="三级指导教师":
```

```
        return 4
```

```
    else:
```

```
        return 2
```

```
teacher["Capacity"]=(
```

```
    teacher["Ability"]
```

```
    .apply(capacity)
```

```
)
```

```
# =====
```

```
# 读取队伍
```

```
# =====
```

```
raw=pd.read_excel(  
    "附件 2 建模队信息.xls",  
    header=None  
)
```

```
team=raw.iloc[3:].copy()
```

```
team_data=pd.DataFrame()
```

```
team_data["TeamID"]=team[0]
```

```
team_data["Major1"]=team[3]  
team_data["Major2"]=team[9]  
team_data["Major3"]=team[15]
```

```
team_data["Exp1"]=team[5]  
team_data["Exp2"]=team[11]  
team_data["Exp3"]=team[17]
```

```
team_data["Rank1"]=team[6]  
team_data["Rank2"]=team[12]  
team_data["Rank3"]=team[18]
```

```
# 删除空队号
```

```
team_data=team_data.dropna(  
    subset=["TeamID"]  
)
```

```
# 只保留数字队号
```

```
team_data["TeamID"] = pd.to_numeric(
```

```

        team_data["TeamID"],
        errors="coerce"
    )

# 删除无法转换的行
team_data = team_data.dropna(
    subset=["TeamID"]
)

# 转整数

team_data["TeamID"] = (
    team_data["TeamID"]
    .astype(int)
)
print(
    "有效队伍数量:",
    len(team_data)
)

print(
    team_data.tail()
)

# =====
# 队伍评分
# =====

def rank_score(x):

    try:

        a,b=x.split("/")

        return 1-float(a)/float(b)

    except:

        return 0

```

```
for c in [
    "Rank1",
    "Rank2",
    "Rank3"
]:

    team_data[c+"Score"]=(
        team_data[c]
        .apply(rank_score)
    )
```

```
team_data["RankScore"]=(

team_data["Rank1Score"]
+
team_data["Rank2Score"]
+
team_data["Rank3Score"]

)/3
```

```
team_data["ExpScore"]=(

team_data["Exp1"].astype(float)
+
team_data["Exp2"].astype(float)
+
team_data["Exp3"].astype(float)

)/6
```

```
team_data["TeamScore"]=(
0.5*team_data["RankScore"]
+
0.5*team_data["ExpScore"]
)
```



```

# =====
# 研究方向匹配
# =====

def research_match(a,b):

    dic={

        "算法":["计算机","电科"],
        "数据":["计算机","应数"],
        "统计":["应数"],
        "优化":["数学","应数"],
        "图":["计算机"],
        "模糊":["数学"],
        "微分":["数学"]

    }

    for k,v in dic.items():

        if k in str(a):

            for m in v:

                if m in str(b):

                    return 1

            return 0

    return 0

# =====
# 双向匹配评分
# =====

```

```

match=[]

for _t in teacher.iterrows():

    for _team in team_data.iterrows():

        major=(

            str(team.Major1)
            +
            str(team.Major2)
            +
            str(team.Major3)

        )

        M=research_match(
            t["主要研究方向"],
            major
        )

        score=(

            0.5*t.AbilityScore
            +
            0.3*team.TeamScore
            +
            0.2*M

        )

        match.append(

            [
                t["序号"],
                team.TeamID,
                score
            ]
        )

```

```
]
```

```
)
```

```
score=pd.DataFrame(
```

```
match,
```

```
columns=[
```

```
"Teacher",
```

```
"Team",
```

```
"Score"
```

```
]
```

```
)
```

```
# =====
```

```
# 教师排序
```

```
# =====
```

```
teacher_pref={}
```

```
for tid in teacher["序号"]:
```

```
    temp=score[
```

```
        score.Teacher==tid
```

```
    ]
```

```
    teacher_pref[tid]=list(
```

```
        temp.sort_values(
```

```
            "Score",
```

```
            ascending=False
```

```
        )
```

```
        ["Team"]
```

)

```
# =====  
# 容量约束稳定匹配  
# =====
```

```
# 每个教师当前指导队伍
```

```
teacher_used={}
```

```
for t in teacher["序号"]:  
    teacher_used[t]=[]
```

```
# 每个队伍当前教师
```

```
team_owner={}
```

```
# 教师申请列表
```

```
teacher_queue=[]
```

```
for t in teacher["序号"]:
```

```
    cap=teacher.loc[  
        teacher["序号"]==t,  
        "Capacity"  
    ].values[0]
```

```
    # 一个教师拥有多个申请机会  
    for _ in range(cap):
```

```
        teacher_queue.append(t)
```

```
while len(teacher_queue)>0:

    t=teacher_queue.pop(0)

    # 已经达到容量
    capacity_t=teacher.loc[
        teacher["序号"]==t,
        "Capacity"
    ].values[0]

    if len(teacher_used[t])>=capacity_t:
        continue

    # 没有候选队伍

    if len(teacher_pref[t])==0:
        continue

    # 教师申请自己最喜欢的队伍

    team_id=teacher_pref[t].pop(0)

    # 如果队伍没有老师

    if team_id not in team_owner:

        team_owner[team_id]=t

        teacher_used[t].append(team_id)

    else:
```

```
old_teacher=team_owner[team_id]
```

```
old_score=score[
```

```
    (score.Teacher==old_teacher)  
    &  
    (score.Team==team_id)
```

```
].Score.values[0]
```

```
new_score=score[
```

```
    (score.Teacher==t)  
    &  
    (score.Team==team_id)
```

```
].Score.values[0]
```

```
# 新老师更适合
```

```
if new_score>old_score:
```

```
    team_owner[team_id]=t
```

```
    teacher_used[old_teacher].remove(  
        team_id  
    )
```

```
    teacher_used[t].append(  
        team_id  
    )
```

```
# 原教师重新申请
```

```

        teacher_queue.append(
            old_teacher
        )

    else:

        # 队伍拒绝，教师继续申请

        teacher_queue.append(
            t
        )

# =====
# 检查未分配队伍
# =====

unassigned=[]

for team_id in team_data["TeamID"]:

    if team_id not in team_owner:

        unassigned.append(team_id)

print(
    "未分配队伍:",
    len(unassigned)
)

# =====
# 如果还有剩余队伍
# 贪心补充分配
# =====

```

```
for team_id in unassigned:
```

```
    temp=score[
        score.Team==team_id
    ].sort_values(
        "Score",
        ascending=False
    )
```

```
    for _,r in temp.iterrows():
```

```
        tid=r.Teacher
```

```
        cap=teacher.loc[
            teacher["序号"]==tid,
            "Capacity"
        ].values[0]
```

```
        if len(teacher_used[tid])<cap:
```

```
            team_owner[team_id]=tid
```

```
            teacher_used[tid].append(
                team_id
            )
```

```
            break
```

```
# =====
# 生成最终结果
# =====
```

```
final=[]
```



```
for team_id,tid in team_owner.items():
```

```
    s=score[

        (score.Teacher==tid)
        &
        (score.Team==team_id)

    ].Score.values[0]
```

```
    final.append(
        [
            tid,
            team_id,
            s
        ]
    )
```

```
final=pd.DataFrame(
```

```
    final,

    columns=[
        "教师编号",
        "队伍编号",
        "匹配得分"
    ]
```

```
)
```

```
print(final.head(20))
```

```
print(
    "最终分配:",
    len(final),
    "支队伍"
```

```

)

# 教师承担数量统计

count=final.groupby(
    "教师编号"
).size().reset_index(
    name="指导队伍数"
)

print(count)

# 保存结果

final.to_excel(
    "问题四_容量稳定匹配结果.xlsx",
    index=False
)

print(
    "完成，共分配:",
    len(final),
    "支队伍"
)

# =====
#                      可视化分析
# =====

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']    # 黑体
plt.rcParams['axes.unicode_minus'] = False

# =====

```

```
# 图 1 教师指导队伍数量柱状图
# =====
```

```
teacher_count = final.groupby(
    "教师编号"
).size()
```

```
plt.figure(figsize=(12,5))
```

```
plt.bar(
    teacher_count.index,
    teacher_count.values
)
```

```
plt.xlabel("教师编号")
```

```
plt.ylabel("指导队伍数量")
```

```
plt.title(
    "各教师指导队伍数量分布"
)
```

```
plt.xticks(rotation=90)
```

```
plt.grid(axis="y")
```

```
plt.tight_layout()
```

```
plt.savefig(
    "图 1_教师指导数量分布.png",
    dpi=300
)
```

```
plt.show()

# =====
# 图 2 匹配得分分布
# =====

plt.figure(figsize=(8,5))

plt.hist(
    final["匹配得分"],
    bins=20
)

plt.xlabel(
    "匹配得分"
)

plt.ylabel(
    "队伍数量"
)

plt.title(
    "教师-队伍匹配得分分布"
)

plt.grid()

plt.tight_layout()

plt.savefig(
    "图 2_匹配得分分布.png",
    dpi=300
)
```

```
plt.show()
```

```
# =====  
# 图 3 教师等级与指导数量关系  
# =====
```

```
level_data = final.merge(  
    teacher[  
        [  
            "序号",  
            "Ability"  
        ]  
    ],  
    left_on="教师编号",  
    right_on="序号",  
    how="left"  
)
```

```
level_count = level_data.groupby(  
    "Ability"  
)[["队伍编号"].count()
```

```
plt.figure(figsize=(7,5))
```

```
plt.bar(  
    level_count.index,  
    level_count.values  
)
```

```
plt.xlabel(
    "教师等级"
)

plt.ylabel(
    "指导队伍数量"
)

plt.title(
    "不同等级教师指导数量比较"
)

plt.grid(
    axis="y"
)

plt.tight_layout()

plt.savefig(
    "图 3_教师等级指导数量.png",
    dpi=300
)

plt.show()
import matplotlib.pyplot as plt

# 教师等级和指导数量

plot_data = count.merge(
    teacher[["序号", "Ability"]],
    left_on="教师编号",
    right_on="序号"
)
```

```
plt.figure(figsize=(8,5))

data=[]

labels=[]

for level in [
    "一级指导教师",
    "二级指导教师",
    "三级指导教师",
    "四级指导教师"
]:

    temp=plot_data[
        plot_data["Ability"]==level
    ]["指导数量"]

    if len(temp)>0:

        data.append(temp)

        labels.append(level)

plt.boxplot(
    data,
    labels=labels
)

plt.ylabel(
    "指导队伍数量"
)

plt.xlabel(
    "教师等级"
)
```

```
plt.title(  
"不同等级教师指导队伍数量分布"  
)
```

```
plt.grid()
```

```
plt.savefig(  
"教师等级指导数量箱线图.png",  
dpi=300  
)
```

```
plt.show()
```